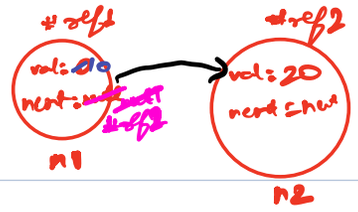
Today's agenda

- ↳ insert in a linkedlist
- ↳ delete in a linkedlist
- ↳ Reverse the linkedlist
- ↳ Find mid
- ↳ floyd cycle.

AlgoPrep

```
int x = 10;
```

```
Public class Node {
    int x;
    String st;

}

Node temp = new Node();
```




```
class Node {
    int val;
    Node next;

    Node (int v1) {
        val = v1;
    }
}
```

```
Node n1 = new Node ();
n1.val = 10;

Node n2 = new Node ();
→ n2.val = 20;
```

```
Print (n1.next.val)
```

```
n1.next = ~~n2.next~~ null;
→ n1.next = ~~n2~~ #ref2;
```

AlgoPrep

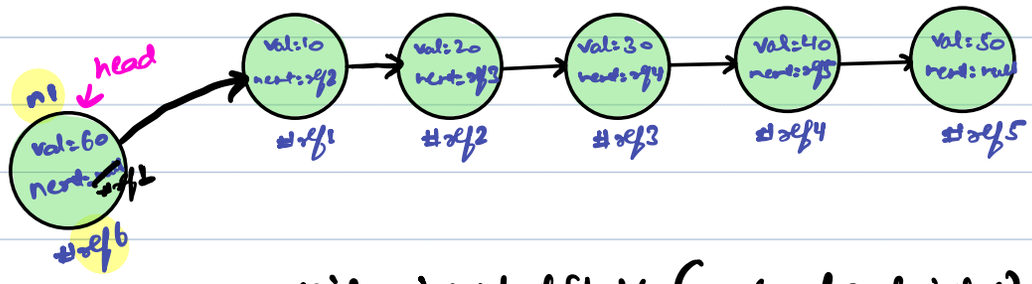
Q) insert in a linkedlist

Print (head.next.next.next.val);

↳ 40

insert at head

↳ v=60




```
void insertatStart (Node head, int v) {
    Node n1 = new Node (v);
    n1.next = head;
    head = n1;
}
```

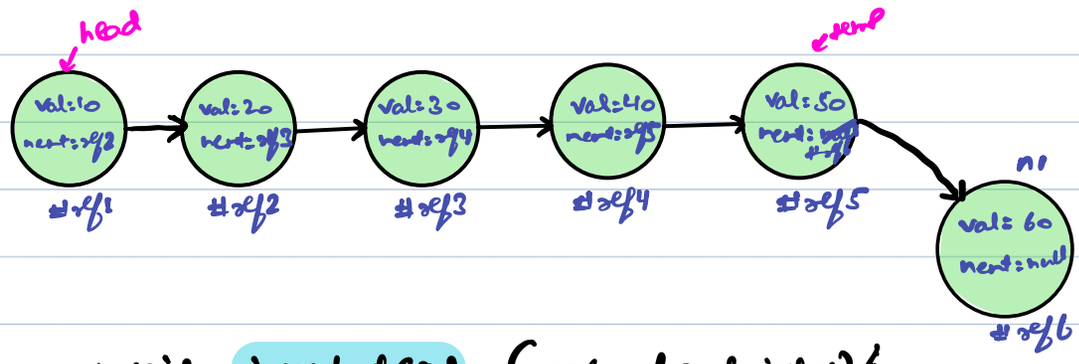
T.C: O(1)

S.C: O(1)

AlgoPrep

insert after last node

↳ v=60




```
void insert at end (node head, int v) {
    node n1 = new node (v);
    node temp = head;
    while (temp.next != null) {
        temp = temp.next;
    }
    temp.next = n1;
}
```

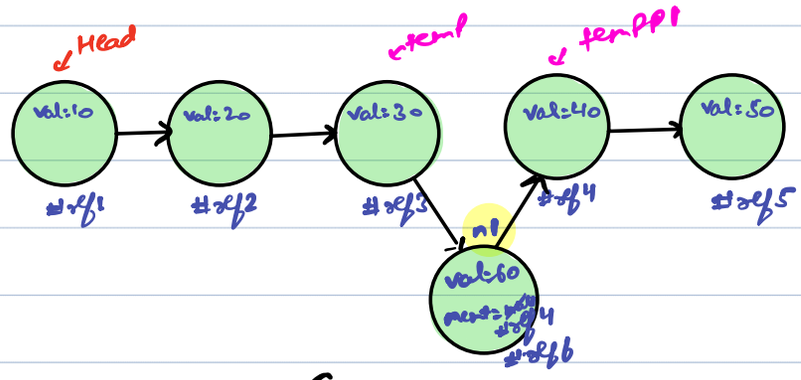
T.C: O(N)

S.C: O(1)

AlgoPrep

Head → null

"insert at index" → K=3 , v=60




```
void insert (Node head, int v, int k) {
    Node n1 = new Node(v);
    Node temp = head;
    for (int i=1; i<=k-1; i++) {
        temp = temp.next;
    }
    Node temPP1 = temp.next;

    temp.next = n1;
    n1.next = temPP1;
}
```

T.C: $O(k) = O(n)$

S.C: $O(1)$

edge cases

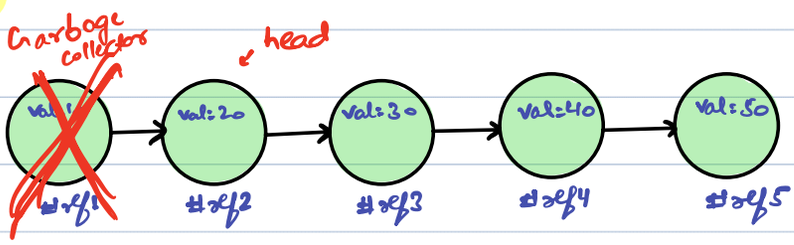
↳ K==0

↳ head==null

Break till 9:40 PM

Q) Delete in a linkedlist

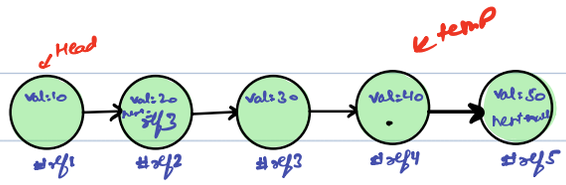
→ delete Head



↳ head = head.next;

AlgoPrep

→ delete last



for 2nd last node
↳ .next.next==null

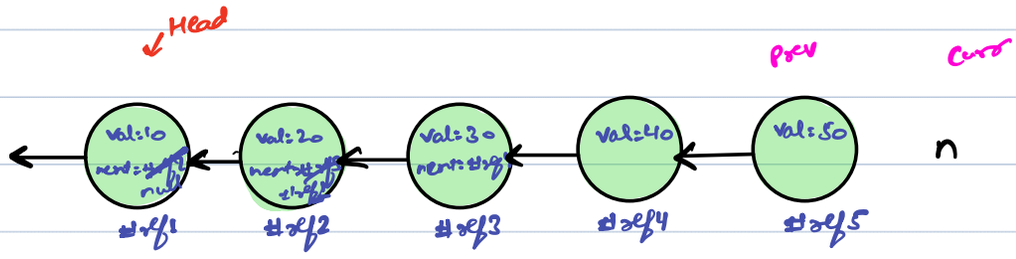
```
deleteLast (node head){
    Node temp = head;
        while (temp.next.next != null) {
            temp = temp.next;
        }

        temp.next = null;
}
```

T.C: O(N)

S.C: O(1)

AlgoPrep

Q) Reverse a linkedlist




```
void reverse (Node head) {

    Node Curr = head;
    Node Prev = null;
    while (Curr != null) {
        Node CurrP1 = Curr.next;
        Curr.next = Prev;
        Prev = Curr;
        Curr = CurrP1;
    }

    head = Prev;
}
```

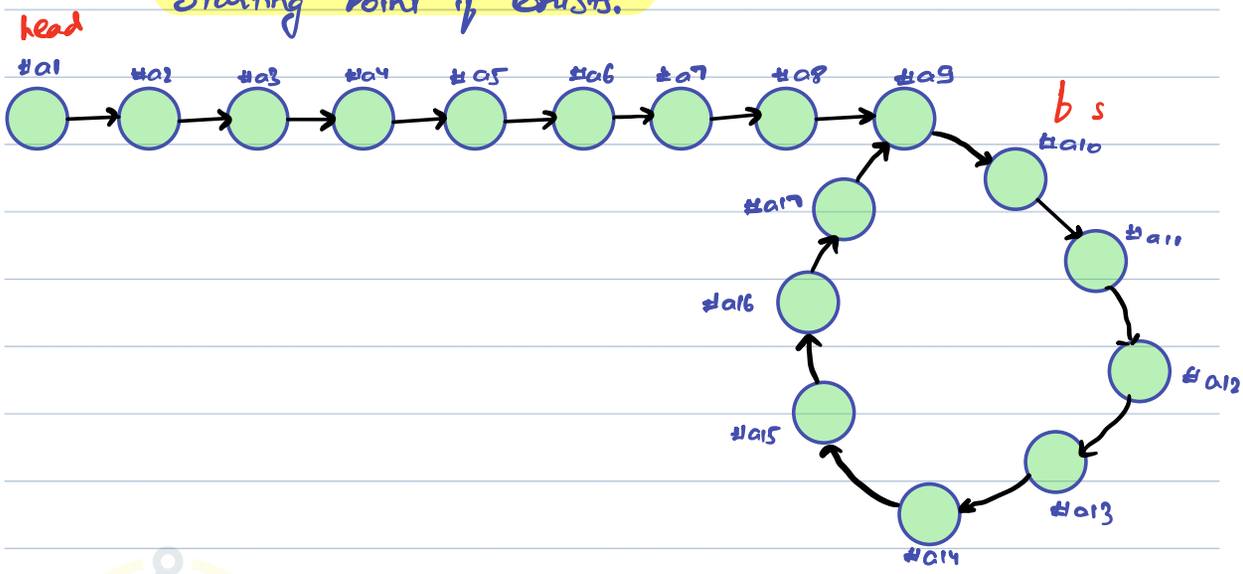
T.C : O(N)

S.C : O(1)

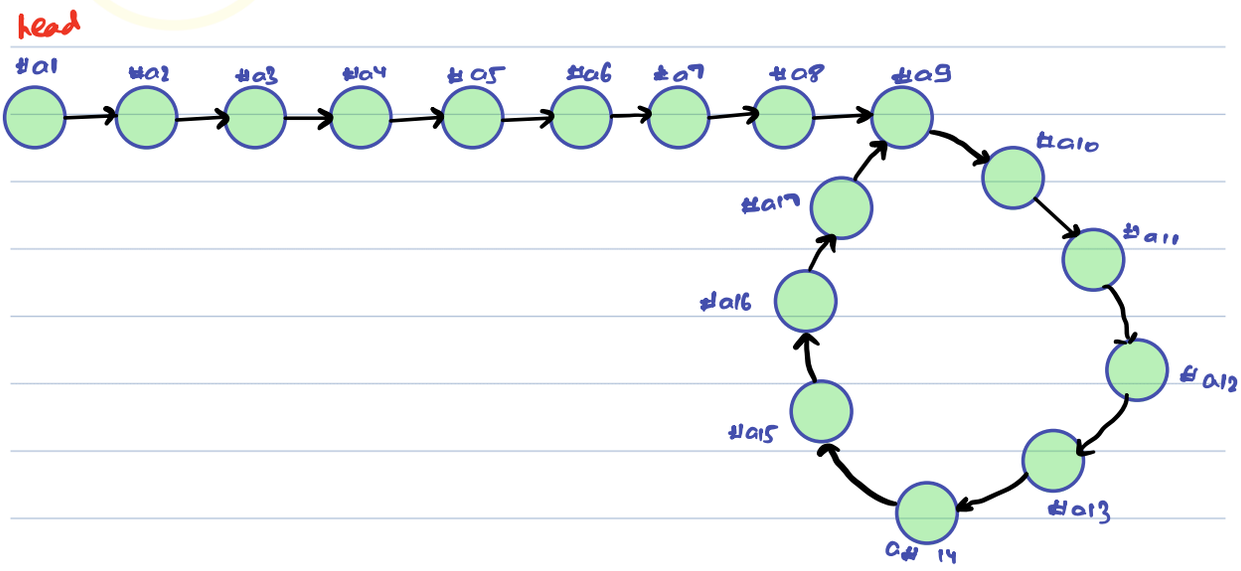
AlgoPrep

floyd cycle

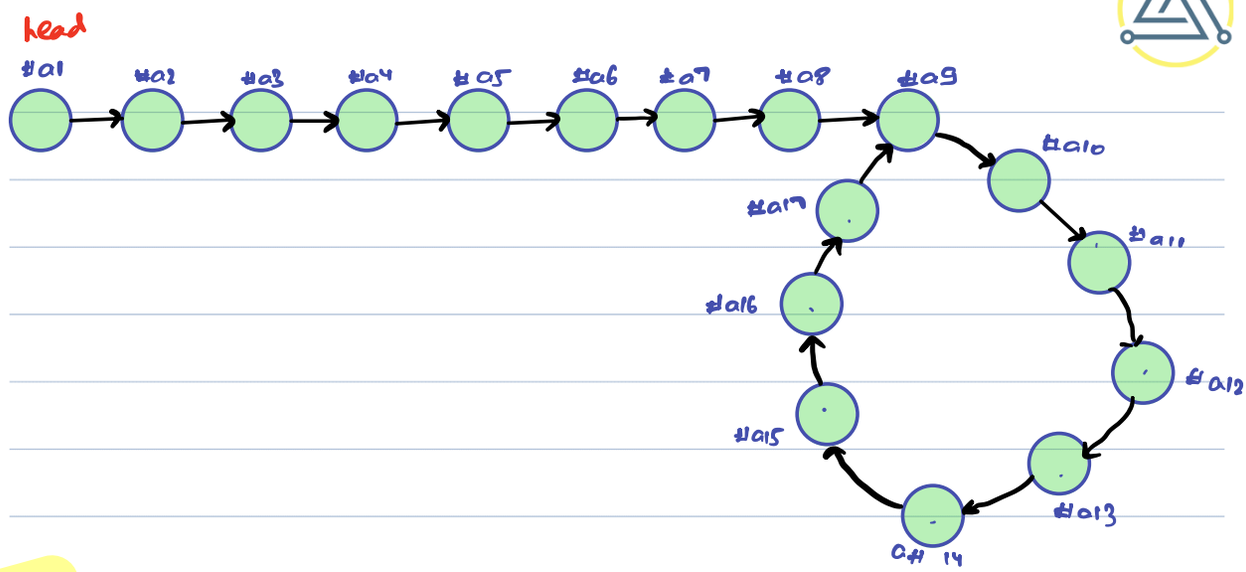
Q) Given a linked list, check for cycle & return the Starting Point if exists.



AlgoPrep

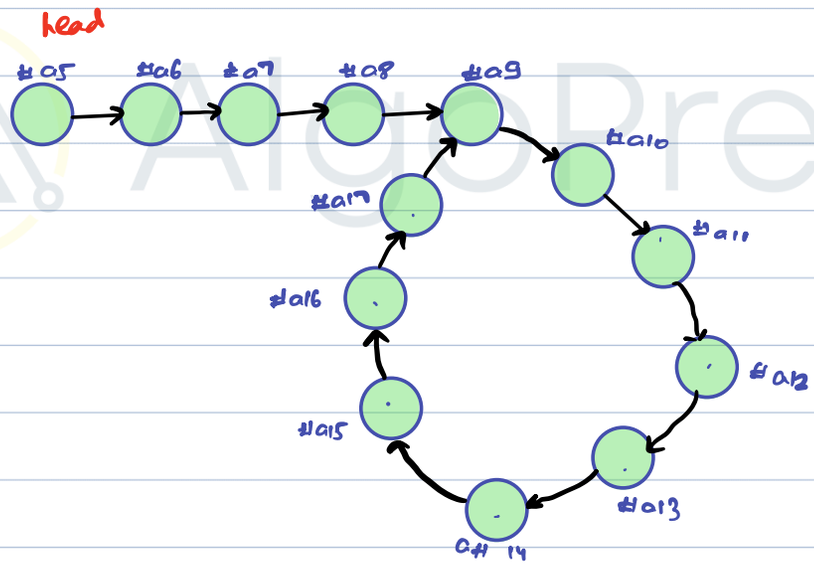


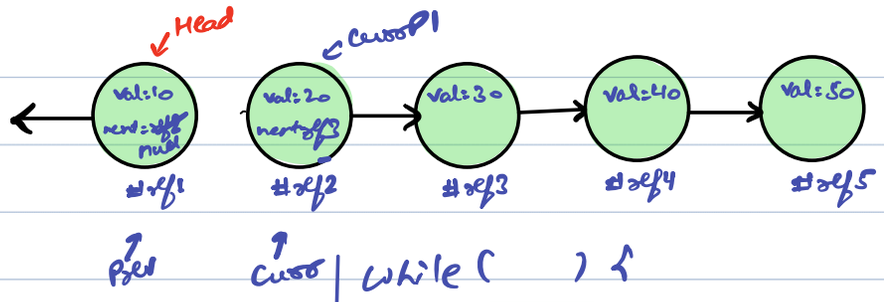
End:



End2:

Chain: 4 Cycle: 9






```
while (    ) {
    Node CurrPl = Curr.next;
    Curr.next = Prev;
      Prev = Curr;
      Curr = CurrPl;
}
```